

Asymptotic Notation & Convergence of Algorithms

Review of complexity and floating point, then analysis of algorithms

Sharareh Sayyad

Washington state University

January 30, 2026

Outline

Review: complexity and floating point

Big- O notation

Convergence of algorithms

Summary

Review of last session

Computational complexity

- ▶ The resources an algorithm needs, as a function of the input size n .
- ▶ **Time**: number of **flops** ($+$, $-$, $\times$, $\div$ on floats).
- ▶ **Space**: extra memory beyond the input.

Floating point

- ▶ Scientific notation in base 2: 64 bits (1 sign, 11 exponent, 52 fraction).
- ▶ $\varepsilon = 2^{-52} \approx 2.2 \times 10^{-16}$: about 16 significant digits; every flop is rounded.
- ▶ Gaps grow with size: 2.2×10^{-16} near 1, but 2 near 10^{16} .

Pitfalls

never test floats with `==`; small + large loses the small; subtracting nearly equal numbers destroys digits.

Today: **Big-O** makes “how the time grows” precise, and **convergence** meets the floating point floor of 10^{-16} .

Big-O: what it tells you

If the input gets bigger, roughly how much more work is needed?

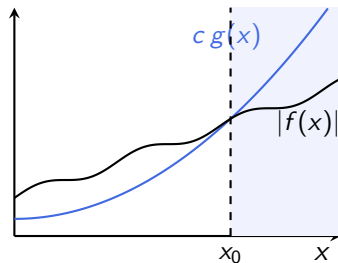
(for large inputs, ignoring constant factors such as a faster computer)

Definition

$f = O(g)$ as $x \rightarrow \infty$ if there are constants $c > 0$ and $x_0 \geq 0$ with

$$|f(x)| \leq c g(x) \quad \text{for all } x \geq x_0.$$

In words: from some point x_0 on, f never exceeds a fixed multiple of g , so f *grows no faster than* g . It is an **upper bound**.



In the shaded region, $|f|$ stays below $c g$.

Simplifying Big-O: keep only what matters

Rule 1: drop constant factors

- ▶ $O(5x) = O(x)$
- ▶ $O(c) = O(1)$
- ▶ $O(cf(x)) = O(f(x))$

Rule 2: keep only the fastest-growing term

- ▶ $3n^2 + 10n + 50 = O(n^2)$
- ▶ $n + \log n = O(n)$
- ▶ $2^n + n^3 = O(2^n)$

Why?

Big-O is about **large** n . There, constants do not change how the work grows, and the fastest-growing term is much bigger than all the others.

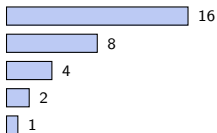
Cheat sheet: common Big- O patterns

code pattern	Python snippet	Big- O	example algorithm
no loops	<code>x = a[5]</code>	$O(1)$	reading an array element by index
halve the input each step	<code>while n > 1: n //= 2</code>	$O(\log n)$	binary search
one loop over n items	<code>for x in a: total += x</code>	$O(n)$	going through an array once
repeat a halving loop n times	<code>for i in range(n): k = n while k > 1: k //= 2</code>	$O(n \log n)$	merge sort; Python's <code>sorted(a)</code>
two nested loops	<code>for i in range(n): for j in range(n):</code>	$O(n^2)$	bubble sort; every cell of an $n \times n$ grid
try every subset	<code>for k in range(2**n):</code>	$O(2^n)$	listing all subsets of a list

Green grows slowly, red grows fast. The $n \log n$ row is the $O(n)$ loop around the $O(\log n)$ loop.

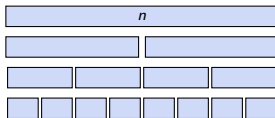
Why these growth rates?

$$O(\log n)$$



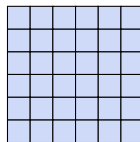
Each step keeps half: 16 items need 4 halvings, $\log_2 16 = 4$.

$$O(n \log n)$$



Merge sort: $\log_2 n$ levels of halving, and combining costs n per level.

$$O(n^2)$$



Nested loops visit all $n \times n$ pairs: twice the input, four times the work.

$O(1)$ and $O(n)$ need no picture: a fixed amount of work, or a fixed amount per element.

Proving Big-O vs. measuring it

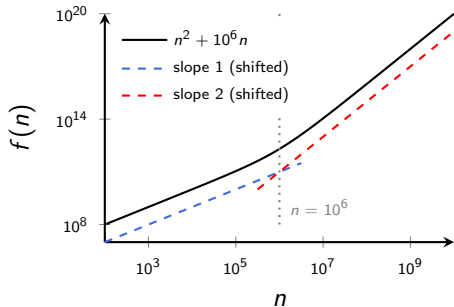
- ▶ Big- O is a statement about **all** $x \geq x_0$. It is **proved**: find c, x_0 (or use a limit).
- ▶ Experiments (timings, loglog slopes, the doubling factor) look at **finitely many** n . They can **check** a prediction, but not prove it.

A measurement that misleads

$f(n) = n^2 + 10^6 n$ is $O(n^2)$ and **not** $O(n)$, yet near $n = 1000$:

- ▶ doubling factor $f(2n)/f(n) = 2.00$;
- ▶ loglog slope on $100 \dots 1600$: 1.00.

Both suggest $O(n)$. The n^2 term only wins for $n > 10^6$ (slope 2.0 near 10^8).



The doubling factor is only reliable for polynomial growth, and only once n is large enough: $\log n$ shows up as $+1$, and 2^n gives no constant factor at all.

Complexity of an algorithm: maximum

```
def maximum(x):  
    xmax = -np.inf  
    for xi in x:  
        if xi > xmax:  
            xmax = xi  
    return xmax
```

- ▶ One loop over the n entries, constant work per iteration.

- ▶ **Time:** $O(n)$. **Space:** $O(1)$ extra.

“Runs in $O(n^2)$ time” means: the run time scales, at worst, quadratically with n .

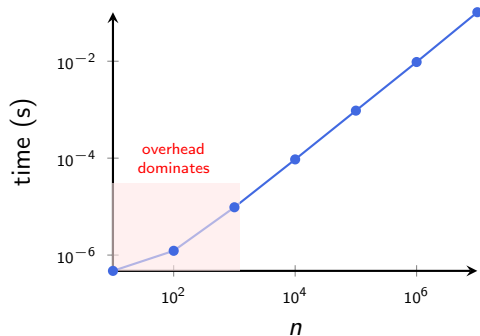
To **check** this prediction, time it and plot on loglog axes. If $t(n)$ grows like n^a ,

$$t(n) \sim c n^a \Rightarrow \log t \sim a \log n + \log c,$$

so the **slope is the exponent a** :

```
coeff = np.polyfit(np.log(ns),  
                  np.log(ts), 1)  
coeff[0]      # estimated exponent
```

Measuring maximum on a loglog plot



fit over	slope
all n (10 to 10^7)	0.92
$n \geq 10^4$ only	1.01

- ▶ For small n the fixed cost of calling the function dominates, which flattens the curve.
- ▶ **Fit only the large- n part.** Then the slope ≈ 1 **agrees** with the prediction $O(n)$.

(measured in pure Python; your times will differ, the slope should not)

Exercise: why semilogy for exponential growth?

If $t(n) \sim c b^n$, then

$$\log t(n) \sim n \log b + \log c.$$

- ▶ On a semilogy plot this is a **straight line**; on normal axes the values span many orders of magnitude.
- ▶ The **slope** is $\log b$, so $b = e^{\text{slope}}$ is the growth factor per extra element.

Example

If the time **doubles** with each extra element, $t(n) \sim c 2^n$: the semilogy plot is a straight line with slope $\log 2 \approx 0.69$, and $e^{0.69} = 2$ recovers the factor.

Compare: polynomial $\Rightarrow$ loglog, slope = exponent. Exponential $\Rightarrow$ semilogy, slope = $\log b$.

Convergence of algorithms

Many numerical algorithms produce better and better approximations x_k of a true solution x_* .

The **error** is

$$\epsilon_k = |x_k - x_*|,$$

and the algorithm **converges** if $\lim_{k \rightarrow \infty} \epsilon_k = 0$.

Our goals

not to prove convergence, but to **verify** it, **measure its rate**, and **compare** algorithms using experimental data.

In practice x_* is **unknown**. Two substitutes:

- ▶ run N iterations and use $\tilde{\epsilon}_k = |x_k - x_N|$;
- ▶ monitor the change $\delta_k = |x_k - x_{k-1}|$ and **stop** when it is small enough.

Floating point floor

δ_k cannot drop below about $\epsilon |x_k| \approx 10^{-16} |x_k|$:
never ask for a smaller tolerance, and always cap the number of iterations.

Rate of convergence: how fast does the error shrink?

The key question: **by how much does one iteration shrink the error?**

name	each iteration ...	example errors	to reach 10^{-6}
sublinear	shrinks the error by less and less: $\epsilon_{k+1}/\epsilon_k \rightarrow 1$	$\epsilon_k = 1/k$: 1, 0.5, 0.33, 0.25, ...	10^6 iterations
linear	shrinks the error by the same factor $C < 1$: $\epsilon_{k+1} \approx C \epsilon_k$	$C = 0.5$: 1, 0.5, 0.25, 0.125, ...	20 iterations
quadratic	squares the error, so the correct digits double: $\epsilon_{k+1} \approx C \epsilon_k^2$	$10^{-1}, 10^{-2}, 10^{-4}, 10^{-8}$	3 iterations

The order q (precise definition)

The convergence has order q if $\epsilon_{k+1}/\epsilon_k^q \rightarrow C$ with $0 < C < \infty$. Linear and sublinear both have $q = 1$ (only C differs: $C < 1$ vs. $C = 1$); quadratic has $q = 2$; any $q > 1$ is called **superlinear**.

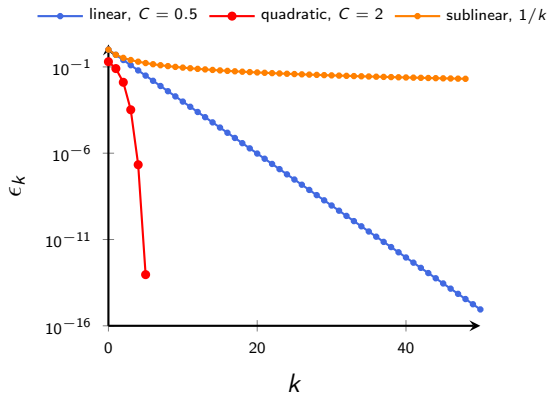
Measuring the rate: semilogy of the error

Taking logarithms of $\epsilon_{k+1} \approx C \epsilon_k^q$:

$$\log \epsilon_{k+1} - q \log \epsilon_k \approx \log C.$$

- ▶ $q = 1$: $\log \epsilon_k$ is a **straight line** with slope $\log C$.
- ▶ $q > 1$: $\log \epsilon_k \rightarrow -\infty$ at an **exponential** rate: the curve bends down.
- ▶ sublinear: the curve **flattens**.

Look at large enough k : the asymptotic behaviour may take many iterations to appear.



Estimating q from data

Asymptotically $\epsilon_{k+1} \approx C\epsilon_k^q$. With $\ell_k = \log \epsilon_k$:

$$\ell_{k+1} \approx \log C + q \ell_k.$$

Subtracting consecutive steps cancels C : the differences $d_k = \ell_{k+1} - \ell_k$ satisfy

$$d_{k+1} \approx q d_k \implies d_k \approx d_0 q^k.$$

So k enters through q^k , and one more log makes it linear:

$$\begin{aligned} \log|\log(\epsilon_{k+1}/\epsilon_k)| &= \log|\log \epsilon_{k+1} - \log \epsilon_k| \\ &= \log|d_k| \approx \log|d_0| + k \log q, \end{aligned}$$

a line in k with **slope** $\log q$.

```
def estimate_q(eps):  
    d = np.diff(np.log(eps))  
    y = np.log(np.abs(d))  
    x = np.arange(len(y))  
    line = np.polyfit(x, y, 1)  
    return np.exp(line[0])
```

sequence	estimated q
sublinear $1/k$	< 1.0
linear ($C = 0.5$)	1.00
quadratic ($C = 2$)	2.00

A rough but easy estimate: it fits a line through *all* the points, including the early, non-asymptotic ones.

Exercise 2: the sublinear estimate depends on the length

$$\epsilon_k = 1/k, \quad k = 1, \dots, n-1:$$

length n	estimated q
50	0.9466
100	0.9720
500	0.9941
1000	0.9970
5000	0.9994
10000	0.9997

- ▶ The estimate approaches $q = 1$ as the sequence gets longer: sublinear convergence has $q = 1$ with $C = 1$.
- ▶ Short sequences are dominated by the early terms, where $\epsilon_{k+1}/\epsilon_k$ is still far from its limit.

Same lesson as for Big- O

An estimate from finitely many data points describes the asymptotic behaviour only once k (or n) is large enough.

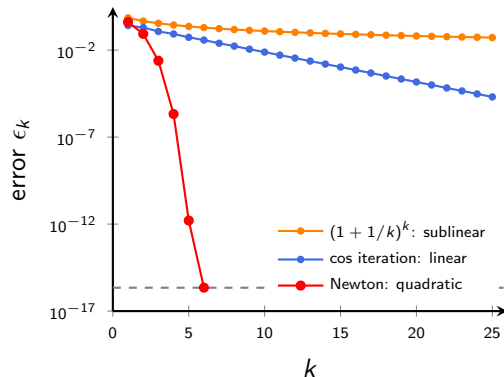
Real algorithms: fixed point vs. Newton

Errors ϵ_k of three real sequences:

- ▶ **linear** (cos iteration): a straight line, slope $\log C$ with $C \approx 0.67$;
- ▶ **quadratic** (Newton for $\sqrt{2}$): bends down;
- ▶ **sublinear** $((1 + 1/k)^k \rightarrow e)$: flattens out.

The floor

Newton reaches $\approx 10^{-16}$ in 5 iterations and cannot go lower: that is machine epsilon.



Summary

Big- O

- ▶ $f = O(g)$: f grows no faster than g (an upper bound).
- ▶ **Prove** with witnesses c, x_0 ; drop constants and smaller terms.
- ▶ Cheat sheet:
 $1 \ll \log n \ll n \ll n \log n \ll n^2 \ll 2^n$.
- ▶ **Check** (not prove) with experiments:
loglog slope for polynomials, semilogy for exponentials; fit only large n .

Convergence

- ▶ Error $\epsilon_k = |x_k - x_*| \rightarrow 0$; stop when $\delta_k = |x_k - x_{k-1}|$ is small.
- ▶ Order q : linear ($q = 1, C < 1$), sublinear, superlinear, quadratic ($q = 2$).
- ▶ semilogy of ϵ_k : line = linear, bends down = superlinear.
- ▶ Floating point sets a floor near 10^{-16} .